

BỘ ĐỀ KIỂM TRA & ĐÁNH GIÁ: Đánh giá Nâng cao về Thuật toán và Lập trình Thi đấu

PHẦN 1: ĐỀ THI TRẮC NGHIỆM (STUDENT QUIZ SHEET)

Câu 1: Dựa trên mô tả về hai kỳ thi IOI và ICPC, hãy đánh giá nhận định sau:

"Mô hình thi đấu của ICPC, với một máy tính cho ba người, thúc đẩy kỹ năng hợp tác và quản lý tài nguyên tốt hơn so với mô hình thi đấu cá nhân của IOI." Nhận định này đúng hay sai và tại sao?

A. Sai, vì IOI đánh giá năng lực cá nhân một cách thuần túy hơn, và đây mới là thước đo quan trọng nhất của một lập trình viên thi đấu.

B. Sai, vì cả hai cuộc thi đều có các vòng loại khu vực và quốc gia, nên kỹ năng hợp tác đã được rèn luyện từ trước cho tất cả thí sinh.

C. Đúng, nhưng kỹ năng quan trọng hơn mà ICPC rèn luyện là kiến thức toán học sâu rộng hơn, chứ không phải kỹ năng hợp tác.

D. Đúng, vì việc chia sẻ một máy tính bắt buộc các thành viên phải giao tiếp, phân chia công việc và lên chiến lược hiệu quả, những kỹ năng không được nhấn mạnh trong thể thức thi đấu cá nhân của IOI.

Câu 2: Một bài toán yêu cầu xử lý các số nguyên có thể lên tới 100 chữ số và thực hiện các phép cộng trừ cơ bản. Thời gian thực thi cho phép là 2 giây. Dựa trên các điểm mạnh và yếu của các ngôn ngữ lập trình được đề cập, lựa chọn nào sau đây thể hiện một sự đánh đổi chiến lược hợp lý nhất trong bối cảnh một cuộc thi lập trình?

A. Sử dụng C++ và tự cài đặt một thư viện xử lý số lớn để đạt hiệu suất thời gian chạy cao nhất có thể.

B. Sử dụng Java với lớp BigInteger vì nó cân bằng giữa hiệu suất và tính năng có sẵn, mặc dù không phổ biến bằng C++ trong các kỳ thi.

C. Sử dụng Python vì nó có sẵn các thao tác cho số nguyên lớn, chấp nhận hiệu suất có thể chậm hơn một chút so với C++ nhưng tiết kiệm đáng kể thời gian lập trình và giảm rủi ro sai sót.

D. Sử dụng C++ với kiểu long long và thêm các bước kiểm tra tràn số, vì hầu hết các bài toán đều được thiết kế để không cần số quá lớn.

Câu 3: Một lập trình viên cần một cấu trúc dữ liệu để lưu trữ các số nguyên duy nhất. Các thao tác chính bao gồm: thêm phần tử, kiểm tra sự tồn tại của phần tử, và tìm phần tử nhỏ nhất lớn hơn một giá trị x cho trước. Đánh giá lựa chọn giữa `std::set` và `std::unordered_set` trong C++, đâu là lựa chọn tối ưu và tại sao?

A. `std::set` vì nó duy trì các phần tử theo thứ tự, cho phép tìm kiếm hiệu quả phần tử nhỏ nhất lớn hơn x , điều mà `std::unordered_set` không hỗ trợ một cách hiệu quả.

- B. `std::unordered_set` vì nó có thể hiệu quả hơn, và việc tìm phần tử nhỏ nhất lớn hơn x có thể được thực hiện bằng cách duyệt qua tất cả các phần tử.
- C. `std::unordered_set` vì các thao tác thêm và kiểm tra tồn tại có độ phức tạp trung bình là $O(1)$, nhanh hơn $O(\log n)$ của `std::set`.
- D. Cả hai đều không phù hợp, nên sử dụng một `std::vector` đã được sắp xếp và thực hiện tìm kiếm nhị phân cho mọi thao tác.

Câu 4: Xét đoạn mã C++ sau: `map<string, int> m; cout << m["aybabbu"];`. Dựa trên hành vi của `std::map` được mô tả, hệ quả tiềm ẩn nào của dòng lệnh này có thể gây ra lỗi logic khó tìm trong một bài toán lập trình thi đấu?

- A. Một giá trị rác hoặc không xác định sẽ được in ra, dẫn đến kết quả sai ngẫu nhiên.
- B. Khóa "aybabbu" sẽ được tự động thêm vào map với giá trị mặc định (0 cho int), có thể vô tình làm tăng kích thước map và ảnh hưởng đến logic đếm các phần tử.
- C. Chương trình sẽ ném ra một ngoại lệ (exception) lúc chạy vì truy cập khóa không hợp lệ.
- D. Chương trình sẽ bị lỗi biên dịch vì không thể in ra giá trị của một khóa không tồn tại.

Câu 5: Một lập trình viên muốn xóa chỉ một phiên bản của số 5 khỏi một `multiset<int> s` đang chứa ba số 5. Dựa trên các ví dụ được cung cấp, phương pháp nào sau đây là đúng và hiệu quả?

- A. `s.erase(s.find(5));`
- B. `s.pop(5);`
- C. `s.erase(5);`
- D. Duyệt qua `multiset`, tìm một số 5 và xóa nó bằng iterator.

Câu 6: Đánh giá các ưu và nhược điểm của việc sử dụng `#include <bits/stdc++.h>` trong lập trình thi đấu so với một dự án kỹ thuật phần mềm truyền thống.

- A. Nó chỉ hữu ích cho người mới bắt đầu; các lập trình viên có kinh nghiệm sẽ luôn include từng thư viện riêng lẻ để tối ưu hóa hiệu suất.
- B. Nó không nên được sử dụng ở bất cứ đâu vì đây là một tính năng không chuẩn của trình biên dịch g++ và có thể không hoạt động trên các hệ thống khác.
- C. Nó phù hợp cho lập trình thi đấu vì giúp viết code nhanh, nhưng không phù hợp cho kỹ thuật phần mềm vì làm tăng thời gian biên dịch và che giấu các dependency của mã nguồn.
- D. Nó luôn là lựa chọn tốt nhất vì giúp tiết kiệm thời gian gõ code và đảm bảo mọi thư viện cần thiết đều có sẵn.

Câu 7: Phân tích lệnh biên dịch C++ sau: `g++ -std=c++11 -O2 -Wall test.cpp -o test`. Nếu loại bỏ cờ `-O2`, hậu quả có khả năng xảy ra cao nhất đối với chương trình trong một kỳ thi là gì?

- A. Chương trình sẽ không tuân thủ chuẩn C++11, gây ra lỗi với các tính năng mới.
- B. Chương trình sẽ không hiển thị các cảnh báo về lỗi tiềm ẩn, làm tăng nguy cơ có lỗi logic.
- C. Chương trình sẽ chạy chậm hơn đáng kể và có thể bị "Time Limit Exceeded" (Quá thời gian) cho các test case lớn.
- D. Chương trình sẽ không biên dịch được vì thiếu tối ưu hóa.

Câu 8: Để tạo một hàng đợi ưu tiên (priorityqueue) trong C++ có khả năng tìm và xóa phần tử nhỏ nhất một cách hiệu quả, cần phải thực hiện sửa đổi nào so với khai báo mặc định, và tại sao?

- A. Không cần sửa đổi, chỉ cần lấy phần tử ở cuối hàng đợi thay vì ở đầu.
- B. Thêm các số âm vào hàng đợi ưu tiên mặc định, sau đó lấy giá trị đối của phần tử lớn nhất.
- C. Sử dụng `std::set` thay thế, vì nó luôn giữ phần tử nhỏ nhất ở `begin()`.
- D. Sử dụng `priorityqueue<int, vector<int>, greater<int>>` để thay đổi bộ so sánh mặc định, biến nó thành một min-heap.

Câu 9: Trong công thức quy hoạch động để tính khoảng cách chỉnh sửa (edit distance) $distance(a,b)$, thành phần $distance(a-1, b-1) + cost(a,b)$ đại diện cho việc phân tích tình huống nào?

- A. Chèn một ký tự vào cuối chuỗi thứ nhất để nó khớp với chuỗi thứ hai.
- B. Xóa ký tự cuối cùng của chuỗi thứ nhất để nó khớp với tiền tố của chuỗi thứ hai.
- C. Xem xét ký tự cuối cùng của cả hai chuỗi: nếu chúng giống nhau thì không tốn chi phí, nếu khác nhau thì thực hiện một thao tác sửa đổi.
- D. Hoán vị hai ký tự cuối cùng của chuỗi thứ nhất.

Câu 10: Áp dụng quy trình xây dựng mã Prüfer được mô tả, mã Prüfer cho cây có nhãn với 6 đỉnh và các cạnh 1-4, 2-4, 3-5, 4-5, 5-6 là gì?

- A. [4, 5, 6, 2]
- B. [4, 4, 5, 6]
- C. [5, 5, 4, 4]
- D. [4, 4, 5, 5]

Câu 11: Một bài toán yêu cầu lưu trữ trạng thái của 1 triệu công tắc (bật/tắt) và thường xuyên thực hiện các phép toán bitwise (AND, OR, XOR) trên toàn bộ tập

hợp công tắc. Đánh giá và chọn cấu trúc dữ liệu được mô tả trong tài liệu mang lại hiệu quả cao nhất về cả bộ nhớ và tốc độ tính toán cho tác vụ này.

- A. `vector<bool>` vì nó được chuyên biệt hóa để lưu trữ các giá trị boolean.
- B. `bitset<1000000>` vì nó chỉ sử dụng 1 bit cho mỗi phần tử và hỗ trợ các toán tử bitwise hiệu quả trên toàn bộ tập hợp.
- C. Một mảng `int[]` với mỗi `int` đại diện cho 32 công tắc để tận dụng các phép toán bitwise của CPU.
- D. `deque<char>` vì nó cho phép thay đổi kích thước hiệu quả và có thể lưu trữ 0 hoặc 1.

Câu 12: Tại sao `""` được cho là hoạt động nhanh hơn `endl` trong C++ khi lập trình thi đấu, dựa trên lý do kỹ thuật được đề cập trong tài liệu?

- A. Vì `endl` luôn thực hiện một thao tác flush (đẩy bộ đệm), buộc luồng xuất phải ghi ra ngay lập tức, gây ra sự chậm trễ không cần thiết.
- B. Vì `""` là một phần của ngôn ngữ C gốc nên được tối ưu hóa tốt hơn bởi trình biên dịch so với `endl` của C++.
- C. Vì `endl` là một hàm, trong khi `""` là một hằng ký tự, nên việc gọi hàm tốn nhiều chi phí hơn.
- D. Vì `endl` không tương thích với `ios::syncwith_stdio(0);` và `cin.tie(0);`, làm mất tác dụng của việc tối ưu hóa I/O.

Câu 13: Một bài toán yêu cầu duy trì một tập hợp các số và thường xuyên phải trả lời truy vấn "phần tử nhỏ thứ k trong tập hợp là gì?". Đánh giá các cấu trúc dữ liệu được đề cập, lựa chọn nào là phù hợp nhất để giải quyết hiệu quả yêu cầu này?

- A. `std::vector`, sau mỗi lần thêm phần tử thì sắp xếp lại và truy cập bằng chỉ số `v[k-1]`.
- B. `indexed_set` (policy-based data structure), vì nó được thiết kế đặc biệt để có thể truy cập các phần tử theo chỉ số như trong một mảng đã sắp xếp.
- C. `std::set`, vì nó giữ các phần tử được sắp xếp và có thể duyệt đến phần tử thứ k.
- D. `priorityqueue`, bằng cách lấy ra k phần tử khỏi hàng đợi.

Câu 14: Biến thể của thuật toán tìm kiếm nhị phân dùng để tìm giá trị cực đại của một hàm (tăng rồi giảm) có một ràng buộc quan trọng được đề cập. Ràng buộc đó là gì và tại sao nó lại cần thiết?

- A. Các giá trị liên tiếp của hàm không được phép bằng nhau. Điều này cần thiết để thuật toán biết cách tiếp tục tìm kiếm (dịch chuyển sang trái hay phải).
- B. Miền giá trị của hàm phải là số nguyên. Điều này cần thiết để các phép so sánh $f(x) < f(x+1)$ có ý nghĩa.
- C. Số lượng phần tử tìm kiếm phải là lũy thừa của 2. Điều này cần thiết để việc chia đôi khoảng tìm kiếm luôn chính xác.

D. Hàm phải là hàm liên tục. Điều này cần thiết để đảm bảo không có bước nhảy đột ngột làm mất đỉnh cực đại.

Câu 15: Một giải pháp cho bài toán trong kỳ thi ICPC đã vượt qua tất cả các ví dụ kiểm thử (sample tests) nhưng lại bị từ chối khi nộp lên hệ thống chấm. Giả sử logic thuật toán là đúng với các bộ dữ liệu nhỏ, nguyên nhân thất bại có khả năng cao nhất là gì, dựa trên tiêu chí chấm điểm của ICPC?

A. Thuật toán chỉ đúng cho các trường hợp đặc biệt có trong ví dụ, nhưng sai ở một trường hợp biên (edge case) nào đó.

B. Thuật toán không đủ hiệu quả và chạy quá giới hạn thời gian cho phép (Time Limit Exceeded) với các bộ dữ liệu kiểm thử lớn và ẩn.

C. Định dạng đầu ra có một sai sót nhỏ, ví dụ như thừa một khoảng trắng, mà các ví dụ kiểm thử không phát hiện được.

D. Đội đã sử dụng sai ngôn ngữ lập trình, ví dụ như Python thay vì C++.

PHẦN 2: ĐÁP ÁN & GIẢI THÍCH CHI TIẾT (ANSWER KEY & EXPLANATIONS)

Câu 1 (Khó): Dựa trên mô tả về hai kỳ thi IOI và ICPC, hãy đánh giá nhận định sau: "Mô hình thi đấu của ICPC, với một máy tính cho ba người, thúc đẩy kỹ năng hợp tác và quản lý tài nguyên tốt hơn so với mô hình thi đấu cá nhân của IOI." Nhận định này đúng hay sai và tại sao?

Đáp án đúng: **D**

Giải thích chi tiết: Tài liệu mô tả ICPC là cuộc thi đồng đội với ba sinh viên và chỉ một máy tính, khác với IOI là thi cá nhân. Việc phải chia sẻ một tài nguyên quan trọng và giới hạn (máy tính) trong môi trường áp lực thời gian cao buộc các thành viên trong đội ICPC phải hợp tác chặt chẽ, phân công nhiệm vụ (ai code, ai suy nghĩ giải thuật, ai gỡ lỗi) và quản lý thời gian sử dụng máy tính một cách chiến lược. Đây là một bài kiểm tra trực tiếp về kỹ năng làm việc nhóm. Ngược lại, thể thức thi cá nhân của IOI tập trung hoàn toàn vào năng lực giải quyết vấn đề của một người. Do đó, phương án A phân tích chính xác hệ quả này. Phương án B đưa ra một quan điểm giá trị ("quan trọng nhất") không được chứng minh trong tài liệu. Phương án C thừa nhận ICPC cần nhiều toán hơn nhưng lại hạ thấp tầm quan trọng của kỹ năng hợp tác, vốn là hệ quả trực tiếp của thể thức thi. Phương án D là một suy luận không có cơ sở.

Câu 2 (Khó): Một bài toán yêu cầu xử lý các số nguyên có thể lên tới 100 chữ số và thực hiện các phép cộng trừ cơ bản. Thời gian thực thi cho phép là 2 giây. Dựa trên các điểm mạnh và yếu của các ngôn ngữ lập trình được đề cập, lựa chọn nào sau đây thể hiện một sự đánh đổi chiến lược hợp lý nhất trong bối cảnh một cuộc thi lập trình?

Đáp án đúng: **C**

Giải thích chi tiết: Tài liệu nêu rõ một trong những thế mạnh của Python là nó "chứa các toán tử sẵn có để tính toán với các số nguyên lớn". Trong một cuộc thi có giới hạn thời gian, việc tiết kiệm thời gian lập trình và giảm thiểu lỗi là cực kỳ quan trọng. Mặc dù C++ hiệu quả hơn về mặt thời gian chạy, việc tự cài đặt một thư viện số lớn là rất tốn thời gian và dễ mắc lỗi. Lựa chọn Python là một quyết định chiến lược, đánh đổi một chút hiệu suất thời gian chạy (vốn có thể vẫn đủ trong giới hạn cho phép) để lấy sự nhanh chóng, an toàn và tiện lợi trong quá trình lập trình. Phương án A không thực tế trong thời gian ngắn của cuộc thi. Phương án C có thể là một lựa chọn, nhưng tài liệu nhấn mạnh cụ thể Python cho trường hợp này. Phương án D hoàn toàn sai vì kiểu long long trong C++ không thể chứa số có 100 chữ số.

Câu 3 (Khó): Một lập trình viên cần một cấu trúc dữ liệu để lưu trữ các số nguyên duy nhất. Các thao tác chính bao gồm: thêm phần tử, kiểm tra sự tồn tại của phần tử, và tìm phần tử nhỏ nhất lớn hơn một giá trị x cho trước. Đánh giá lựa chọn giữa `std::set` và `std::unordered_set` trong C++, đâu là lựa chọn tối ưu và tại sao?

Đáp án đúng: **A**

Giải thích chi tiết: Tài liệu chỉ ra rằng `std::set` (dựa trên cây nhị phân cân bằng) "duy trì thứ tự của các phần tử" và có độ phức tạp $O(\log n)$ cho các thao tác. Ngược lại, `std::unordered_set` (dựa trên bảng băm) có độ phức tạp trung bình $O(1)$ nhưng không duy trì thứ tự. Yêu cầu quan trọng của bài toán là "tìm phần tử nhỏ nhất lớn hơn một giá trị x cho trước". Thao tác này đòi hỏi cấu trúc dữ liệu phải được sắp xếp. `std::set` hỗ trợ trực tiếp thao tác này một cách hiệu quả (ví dụ, thông qua hàm `upperbound` hoạt động trên iterator). `std::unordered_set` không thể thực hiện thao tác này hiệu quả; cách duy nhất là duyệt toàn bộ, tốn $O(n)$. Do đó, dù `std::set` chậm hơn một chút cho việc thêm và tìm kiếm, nó là lựa chọn duy nhất đáp ứng tất cả các yêu cầu một cách hiệu quả. Phương án A và D bỏ qua yêu cầu quan trọng về thứ tự. Phương án C không hiệu quả khi thêm phần tử vì phải sắp xếp lại vector.

Câu 4 (Khó): Xét đoạn mã C++ sau: `map<string, int> m; cout << m["aybabbu"];`. Dựa trên hành vi của `std::map` được mô tả, hệ quả tiềm ẩn nào của dòng lệnh này có thể gây ra lỗi logic khó tìm trong một bài toán lập trình thi đấu?

Đáp án đúng: **B**

Giải thích chi tiết: Tài liệu nêu rõ: "Nếu giá trị của một khóa được yêu cầu nhưng map không chứa nó, khóa đó sẽ tự động được thêm vào map với một giá trị mặc định." Trong trường hợp này, `m["aybabbu"]` sẽ thêm khóa "aybabbu" vào map `m` với giá trị mặc định của `int` là 0, sau đó in ra 0. Trong lập trình thi đấu, việc một cấu trúc dữ liệu tự động thay đổi (thêm phần tử) chỉ bằng một thao tác truy vấn (toán tử `[]`) có thể dẫn đến các lỗi logic tinh vi. Ví dụ, nếu lập trình viên chỉ muốn kiểm tra sự tồn tại của khóa nhưng lại vô tình thêm một phần tử mới, điều này có thể làm sai lệch các phép đếm hoặc các vòng lặp duyệt map sau đó. Phương án A và B sai vì đây là hành vi được định nghĩa của `std::map`, không gây lỗi hay ngoại lệ. Phương án D sai vì giá trị mặc định (0) được thêm vào, không phải giá trị rác.

Câu 5 (Khó): Một lập trình viên muốn xóa chỉ một phiên bản của số 5 khỏi một `multiset<int> s` đang chứa ba số 5. Dựa trên các ví dụ được cung cấp, phương pháp nào sau đây là đúng và hiệu quả?

Đáp án đúng: **A**

Giải thích chi tiết: Tài liệu giải thích rõ ràng về hành vi của `erase` trên `multiset`. Khi gọi `s.erase(value)`, nó sẽ "xóa tất cả các phiên bản của một phần tử". Để xóa chỉ một phiên bản, tài liệu cung cấp chính xác đoạn mã: `s.erase(s.find(5));`. Hàm `s.find(5)` trả về một iterator trỏ đến phiên bản đầu tiên của số 5, và việc gọi `erase` với một iterator sẽ chỉ xóa phần tử tại vị trí đó. Do đó, B là câu trả lời đúng. A sẽ xóa cả ba số 5. C sai vì `multiset` không có hàm `pop`. D về mặt logic là đúng nhưng B là cách làm trực tiếp và ngắn gọn được nêu trong tài liệu.

Câu 6 (Khó): Đánh giá các ưu và nhược điểm của việc sử dụng `#include <bits/stdc++.h>` trong lập trình thi đấu so với một dự án kỹ thuật phần mềm truyền thống.

Đáp án đúng: **C**

Giải thích chi tiết: Tài liệu giới thiệu `#include <bits/stdc++.h>` như một cách để "bao gồm toàn bộ thư viện chuẩn" và giúp viết code nhanh. Trong lập trình thi đấu, nơi "chương trình ngắn" và "không cần bảo trì sau cuộc thi", tốc độ viết code là tối quan trọng, làm cho cú pháp này trở nên hữu ích. Tuy nhiên, trong các dự án kỹ thuật phần mềm truyền thống, việc bao gồm toàn bộ thư viện chuẩn sẽ làm tăng đáng kể thời gian biên dịch và gây khó khăn cho việc quản lý và theo dõi các thư viện phụ thuộc (dependency) thực sự của một file mã nguồn. Do đó, phương án B đưa ra một sự đánh giá cân bằng và chính xác về bối cảnh sử dụng của cú pháp này. A là một khẳng định quá mạnh. C đúng về mặt kỹ thuật (không chuẩn) nhưng bỏ qua lợi ích thực tế của nó trong bối cảnh thi đấu. D sai vì nhiều lập trình viên kinh nghiệm vẫn sử dụng nó trong các cuộc thi để tiết kiệm thời gian.

Câu 7 (Khó): Phân tích lệnh biên dịch C++ sau: `g++ -std=c++11 -O2 -Wall test.cpp -o test`. Nếu loại bỏ cờ `-O2`, hậu quả có khả năng xảy ra cao nhất đối với chương trình trong một kỳ thi là gì?

Đáp án đúng: **C**

Giải thích chi tiết: Tài liệu giải thích rằng cờ `-O2` dùng để "tối ưu hóa mã nguồn" (optimizes the code). Tối ưu hóa của trình biên dịch là quá trình biến đổi mã nguồn để tạo ra một chương trình thực thi chạy nhanh hơn và/hoặc nhỏ hơn. Trong lập trình thi đấu, hiệu suất là yếu tố cực kỳ quan trọng. Việc loại bỏ cờ `-O2` có nghĩa là trình biên dịch sẽ không thực hiện các tối ưu hóa này, dẫn đến mã thực thi chạy chậm hơn. Với các bộ dữ liệu đầu vào lớn, sự khác biệt về tốc độ này có thể là yếu tố quyết định giữa việc chương trình chạy kịp trong giới hạn thời gian và việc bị "Time Limit Exceeded". A sai, chương trình vẫn biên dịch được. C là chức năng của cờ `-Wall` (show warnings). D là chức năng của cờ `-std=c++11`.

Câu 8 (Khó): Để tạo một hàng đợi ưu tiên (priorityqueue) trong C++ có khả năng tìm và xóa phần tử nhỏ nhất một cách hiệu quả, cần phải thực hiện sửa đổi nào so với khai báo mặc định, và tại sao?

Đáp án đúng: **D**

Giải thích chi tiết: Tài liệu nêu rõ rằng "Theo mặc định, các phần tử trong một priorityqueue của C++ được sắp xếp theo thứ tự giảm dần", nghĩa là nó hoạt động như một max-heap, cho phép truy cập phần tử lớn nhất qua `top()`. Để hỗ trợ tìm và xóa phần tử nhỏ nhất, tài liệu cung cấp cú pháp chính xác: `priorityqueue<int,vector<int>,greater<int>> q;`. Cú pháp này cung cấp một bộ so sánh khác (`greater<int>`) cho cấu trúc heap bên dưới, khiến nó sắp xếp các phần tử theo thứ tự tăng dần và hoạt động như một min-heap. Phương án A sai vì priorityqueue không cho phép truy cập phần tử cuối. C là một giải pháp thay thế nhưng không trả lời câu hỏi về cách sửa đổi priorityqueue. D là một mẹo phổ biến nhưng B là cách làm chuẩn và trực tiếp được mô tả trong tài liệu.

Câu 9 (Khó): Trong công thức quy hoạch động để tính khoảng cách chỉnh sửa (edit distance) `distance(a,b)`, thành phần `distance(a-1, b-1) + cost(a,b)` đại diện

cho việc phân tích tình huống nào?

Đáp án đúng: C

Giải thích chi tiết: Tài liệu giải thích công thức $\text{distance}(a,b) = \min(\text{distance}(a,b-1)+1, \text{distance}(a-1,b)+1, \text{distance}(a-1,b-1)+\text{cost}(a,b))$. Nó cũng liệt kê ý nghĩa của từng thành phần. Cụ thể, $\text{distance}(a-1,b-1)$ là khoảng cách chỉnh sửa giữa hai chuỗi ngắn hơn $x[0\dots a-1]$ và $y[0\dots b-1]$. Để chuyển từ $\text{distance}(a-1,b-1)$ sang $\text{distance}(a,b)$, ta cần xử lý hai ký tự cuối cùng là $x[a]$ và $y[b]$. Thành phần $\text{cost}(a,b)$ được định nghĩa là 0 nếu $x[a] = y[b]$ (khớp, không cần thao tác) và 1 nếu $x[a] \neq y[b]$ (cần một thao tác sửa đổi). Do đó, $\text{distance}(a-1,b-1) + \text{cost}(a,b)$ đại diện cho chi phí tối ưu để biến đổi $x[0\dots a]$ thành $y[0\dots b]$ bằng cách xử lý hai ký tự cuối cùng như một cặp, hoặc là khớp hoặc là sửa đổi. A tương ứng với $\text{distance}(a,b-1)+1$. B tương ứng với $\text{distance}(a-1,b)+1$. D (hoán vị) không phải là một thao tác cơ bản trong định nghĩa khoảng cách Levenshtein được đưa ra.

Câu 10 (Khó): Áp dụng quy trình xây dựng mã Prüfer được mô tả, mã Prüfer cho cây có nhãn với 6 đỉnh và các cạnh 1-4, 2-4, 3-5, 4-5, 5-6 là gì?

Đáp án đúng: D

Giải thích chi tiết: Quy trình xây dựng mã Prüfer là lặp đi lặp lại việc "loại bỏ chiếc lá có nhãn nhỏ nhất, và thêm nhãn của hàng xóm duy nhất của nó vào mã".

1. Cây ban đầu có các lá là 1, 2, 3, 6. Lá nhỏ nhất là 1. Hàng xóm của 1 là 4. Xóa đỉnh 1. Mã: [4].

2. Cây còn lại có các lá là 2, 3, 6. Lá nhỏ nhất là 2. Hàng xóm của 2 là 4. Xóa đỉnh 2. Mã: [4, 4].

3. Cây còn lại có các lá là 3, 6. Lá nhỏ nhất là 3. Hàng xóm của 3 là 5. Xóa đỉnh 3. Mã: [4, 4, 5].

4. Cây còn lại có các lá là 4, 6. Lá nhỏ nhất là 4. Hàng xóm của 4 là 5. Xóa đỉnh 4. Mã: [4, 4, 5, 5].

Quá trình kết thúc vì mã đã có $n-2 = 4$ phần tử. Mã Prüfer cuối cùng là [4, 4, 5, 5].

Câu 11 (Khó): Một bài toán yêu cầu lưu trữ trạng thái của 1 triệu công tắc (bật/tắt) và thường xuyên thực hiện các phép toán bitwise (AND, OR, XOR) trên toàn bộ tập hợp công tắc. Đánh giá và chọn cấu trúc dữ liệu được mô tả trong tài liệu mang lại hiệu quả cao nhất về cả bộ nhớ và tốc độ tính toán cho tác vụ này.

Đáp án đúng: B

Giải thích chi tiết: Tài liệu giới thiệu bitset với hai lợi ích chính: (1) yêu cầu ít bộ nhớ hơn mảng thông thường (n bit so với $32n$ bit cho mảng int) và (2) các giá trị có thể được thao tác hiệu quả bằng các toán tử bit. Cả hai lợi ích này đều trực tiếp giải quyết các yêu cầu của bài toán: lưu trữ một lượng lớn trạng thái bật/tắt (tiết kiệm bộ nhớ) và thực hiện các phép toán bitwise (hiệu quả tính toán). Do đó, bitset là lựa chọn vượt trội.

Phương án A, vector<bool>, cũng tiết kiệm bộ nhớ nhưng không được đề cập là có các phép toán bitwise trên toàn bộ tập hợp hiệu quả như bitset. Phương án C mô tả cách triển khai thủ công những gì bitset đã làm sẵn. Phương án D hoàn toàn không hiệu quả về bộ nhớ và không được tối ưu cho các phép toán bit.

Câu 12 (Khó): Tại sao "" được cho là hoạt động nhanh hơn endl trong C++ khi lập trình thi đấu, dựa trên lý do kỹ thuật được đề cập trong tài liệu?

Đáp án đúng: A

Giải thích chi tiết: Tài liệu nêu rất rõ ràng và trực tiếp lý do: "Note that the newline "" works faster than endl, because endl always causes a flush operation." Thao tác flush sẽ buộc bộ đệm (buffer) của luồng xuất (output stream) phải được ghi ra thiết bị đầu ra ngay lập tức. Trong lập trình thi đấu, việc xuất ra một lượng lớn dữ liệu thường được thực hiện hiệu quả hơn bằng cách để hệ thống tự quản lý việc flush bộ đệm khi nó đầy, thay vì buộc flush sau mỗi dòng. Việc flush liên tục này gây ra chi phí I/O không cần thiết và làm chậm chương trình. Các phương án khác đưa ra những lý do có vẻ hợp lý nhưng không phải là lý do chính được nêu trong tài liệu.

Câu 13 (Khó): Một bài toán yêu cầu duy trì một tập hợp các số và thường xuyên phải trả lời truy vấn "phần tử nhỏ thứ k trong tập hợp là gì?". Đánh giá các cấu trúc dữ liệu được đề cập, lựa chọn nào là phù hợp nhất để giải quyết hiệu quả yêu cầu này?

Đáp án đúng: B

Giải thích chi tiết: Tài liệu giới thiệu một cấu trúc dữ liệu đặc biệt từ g++ là indexed_set. Điểm đặc biệt của nó được mô tả là "chúng ta có quyền truy cập vào các chỉ số mà các phần tử sẽ có trong một mảng đã sắp xếp". Điều này có nghĩa là nó hỗ trợ thao tác tìm phần tử thứ k (order statistics) một cách hiệu quả (thường là trong thời gian $O(\log n)$). Đây chính xác là yêu cầu của bài toán. Phương án A không hiệu quả, vì để tìm phần tử thứ k trong std::set, bạn phải duyệt k bước từ đầu, tốn thời gian $O(k)$. Phương án B rất không hiệu quả, mỗi lần thêm phần tử phải sắp xếp lại tốn $O(n \log n)$. Phương án D sẽ làm thay đổi cấu trúc dữ liệu (xóa các phần tử) và cũng không hiệu quả.

Câu 14 (Khó): Biến thể của thuật toán tìm kiếm nhị phân dùng để tìm giá trị cực đại của một hàm (tăng rồi giảm) có một ràng buộc quan trọng được đề cập. Ràng buộc đó là gì và tại sao nó lại cần thiết?

Đáp án đúng: A

Giải thích chi tiết: Tài liệu nêu rõ ràng ràng buộc này: "Note that unlike in the ordinary binary search, here it is not allowed that consecutive values of the function are equal. In this case it would not be possible to know how to continue the search." Thuật toán hoạt động dựa trên việc so sánh $f(x)$ và $f(x+1)$ để quyết định xem đỉnh cực đại nằm ở bên trái hay bên phải. Nếu $f(x) = f(x+1)$, thuật toán không thể xác định được liệu nó đang ở phần tăng, phần giảm, hay trên một "cao nguyên" bằng phẳng, do đó không biết nên thu hẹp không gian tìm kiếm về phía nào. Ràng buộc này đảm bảo rằng tại mọi điểm không phải là đỉnh, luôn có một hướng đi lên hoặc đi xuống rõ ràng.

Câu 15 (Khó): Một giải pháp cho bài toán trong kỳ thi ICPC đã vượt qua tất cả các ví dụ kiểm thử (sample tests) nhưng lại bị từ chối khi nộp lên hệ thống chấm. Giả sử logic thuật toán là đúng với các bộ dữ liệu nhỏ, nguyên nhân thất

bại có khả năng cao nhất là gì, dựa trên tiêu chí chấm điểm của ICPC?

Đáp án đúng: **B**

Giải thích chi tiết: Tài liệu mô tả việc chấm điểm trong ICPC: "Một giải pháp cho một bài toán chỉ được chấp nhận nếu nó giải quyết tất cả các trường hợp kiểm thử một cách hiệu quả." Điều này nhấn mạnh tầm quan trọng của hiệu quả (efficiency), tức là độ phức tạp thời gian của thuật toán. Các ví dụ kiểm thử thường có kích thước nhỏ để thí sinh kiểm tra tính đúng đắn của logic cơ bản. Tuy nhiên, các bộ dữ liệu kiểm thử ẩn trên hệ thống chấm thường có kích thước rất lớn để kiểm tra hiệu quả của thuật toán. Do đó, một thuật toán đúng về logic nhưng quá chậm (ví dụ, độ phức tạp $O(n^2)$ thay vì $O(n \log n)$) sẽ vượt qua các ví dụ nhỏ nhưng thất bại trên các bộ dữ liệu lớn do quá thời gian. Đây là kịch bản phổ biến nhất trong lập trình thi đấu. Các phương án khác cũng có thể xảy ra, nhưng B là lý do liên quan trực tiếp đến tiêu chí "hiệu quả" được nhấn mạnh.